



ENT-164 · Intro to Making



Derby Entrepreneurship
Center at Tufts

CLASS 6 · WEEK 6 · FALL 2026

Connectivity with AI

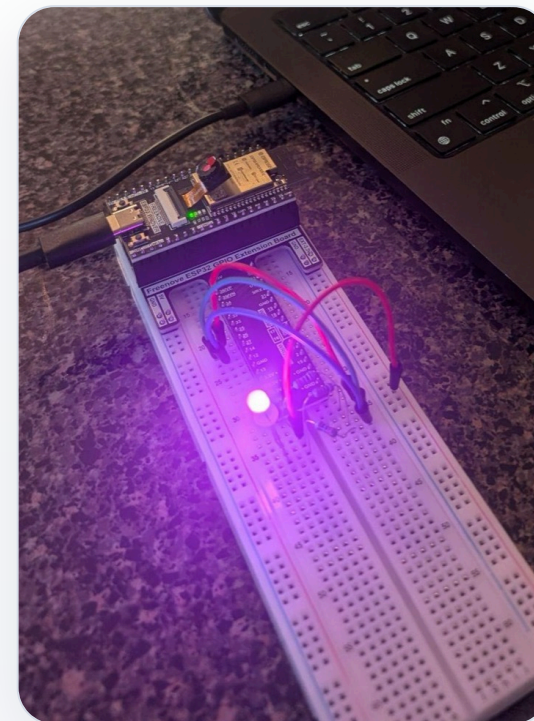
Your ESP32 can serve a web page your phone opens — no app required.
Today we get it on WiFi, register it with Tufts, and build a smart light you
control from the browser.

Thu, Oct 15

JCC 301 / Nolop

No programming required

Bring your ESP32



Before we go **wireless**

You should have all three from last class — tell us now if something is missing.

01

Electronics kit

You've received your electronics kit — the parts you'll wire up today.

02

opencode + MicroPython

opencode is set up and MicroPython is deployed to your ESP32 — it runs your code on the board.

03

AI-assisted electronics

You can already build basic electronics projects with the help of AI.

From your phone **to the internet**

Connect, reach out, and build — the three things we do in class today.

01

Connect to your microcontroller

Reach your device from a phone and from a web browser — no native app needed.

02

Connect to external services

Give your device internet access, higher data rates, and cloud services.

03

Build a Smart Light

Put it all together: an RGB light you control from your phone's browser.

Two ways to connect

Both work. Only one of them skips app development entirely.

Bluetooth

Short-range, device to device

Low-power, short-range communication — like connecting to a phone app. Requires a native mobile app for user interaction.

This is hard. You'd have to build and ship an app.

WiFi

Internet-connected, browser-first

Internet access, higher data rates, and integration with cloud services. Users interact through a simple web browser interface — no app development needed.

Today's approach — the browser is your user interface.

Client, server, or **hybrid**

Three pieces: your device, the services it reaches, and the browser you control it from.



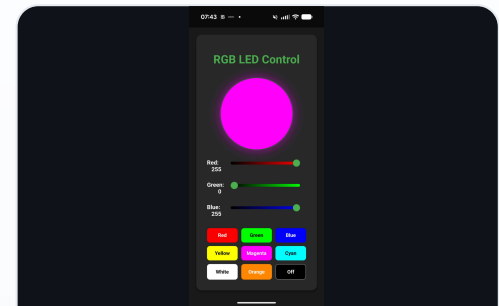
The device

An ESP32 WROVER running MicroPython — the thing that gets online.



External services

Over WiFi, your device can reach APIs and cloud services.



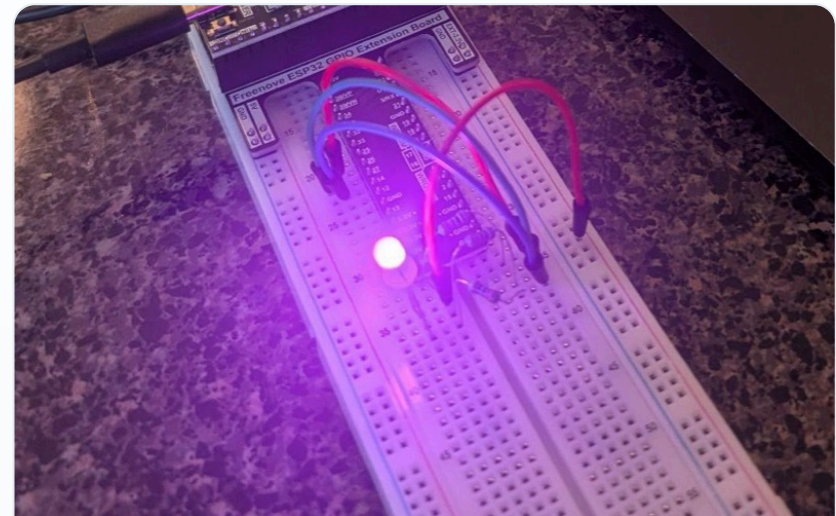
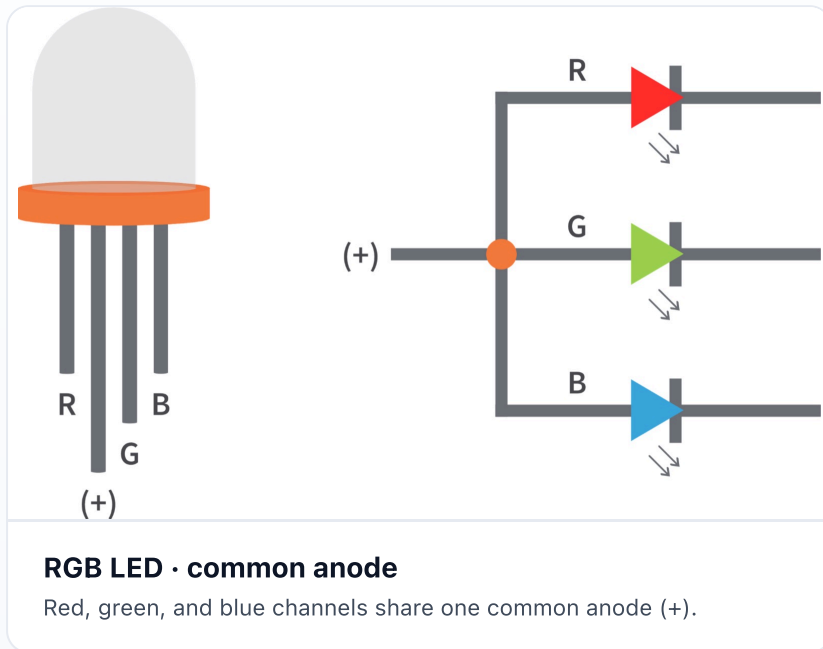
The browser UI

No app to install — the device serves a page your phone opens.

Client — your device reaches out to a service · **Server** — your device serves a page · **Hybrid** — the Smart Light is both.

Wiring the Smart Light

One RGB LED — three colors in a single package, sharing a common anode (+).



On the breadboard

Each channel gets its own pin — the page mixes them into any color.

Ask for it in **plain English**

No programming required — describe the project and let AI write the MicroPython.

THE PROMPT

`"I want to use an ESP32 WROVER with MicroPython to measure the distance with a HC-SR04 sensor and show it on my phone using the Tufts_Wireless open wifi hotspot. Print my ESP32 MAC address on startup."`

WHAT THE AI NEEDS FROM YOU

THE BOARD & LANGUAGE

ESP32 WROVER, MicroPython

THE COMPONENT

HC-SR04 distance sensor

THE GOAL

Measure distance, show it on your phone

THE CONNECTION

The Tufts_Wireless open WiFi hotspot

The MAC line is extra: it prints the address you'll register with Tufts IT — next slide.

Register your ESP32 with Tufts IT

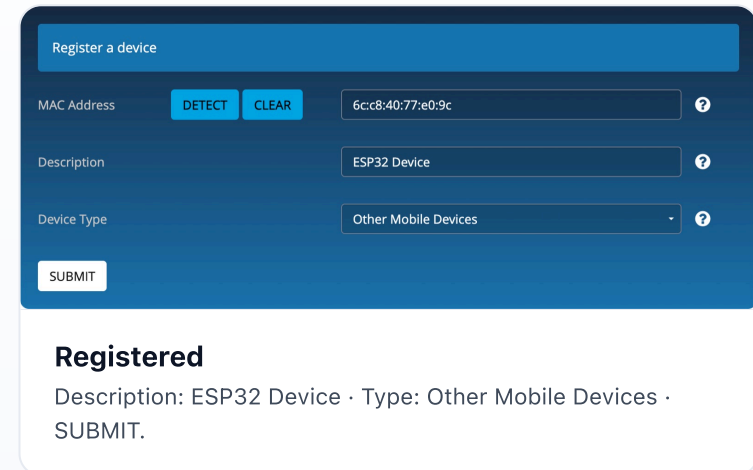
Tufts_Wireless needs your device's MAC address before it will let it join.

01 Run the code with opencode and copy your device's MAC address from what it prints — it looks like `6c:c8:40:77:e0:9c`, but with your own numbers and letters.

02 The rest of the code will fail at this point — you can't reach WiFi yet. That's expected.

03 Go to <https://device-registration.it.tufts.edu>.

04 Clear the default MAC address, then copy your ESP32's MAC from opencode into the field.



The screenshot shows a web form titled "Register a device" with a dark blue header. Below the header, there are three input fields: "MAC Address" with "DETECT" and "CLEAR" buttons, "Description" with a text input, and "Device Type" with a dropdown menu. Each field has a help icon (question mark). The "MAC Address" field contains the text "6cc8:40:77:e0:9c". The "Description" field contains "ESP32 Device". The "Device Type" dropdown is set to "Other Mobile Devices". At the bottom of the form is a "SUBMIT" button. Below the form, the text "Registered" is displayed in bold, followed by "Description: ESP32 Device · Type: Other Mobile Devices · SUBMIT."

Use the IP address, **not the MAC**

opencode prints two addresses. Only one of them belongs in your browser.

Use this

192.168.6.134

Type the IP address opencode prints into your phone's browser, e.g. `http://192.168.6.134`. That opens your device's page.

Not this

6c:c8:40:77:e0:9c

The MAC address was only needed for Tufts IT registration — it will not open a web page.

ATTENTION It may take a few minutes before your device registration is active. If the page doesn't load, wait and refresh.

What just **happened?**

Every tap on your phone becomes a request the ESP32 answers.

A web server starts

1

Your ESP32 runs a web server that renders a web page — no separate computer needed.

The page gets an address

2

The page is accessible at the device's IP address, assigned automatically when it joins Tufts_Wireless.

Your phone renders it

3

Your phone's browser renders the HTML that the ESP32's web server generates.

Requests round-trip

4

Interacting with the page sends a request to the ESP32 (e.g. `http://10.243.94.14/`). It reads the request, calls the sensor, and passes the data back.

Other things to try

Once your device is online, the browser becomes a remote control for anything you build.

01

Visualize sensor data

A dashboard with temperature history charts.

02

Control devices from your phone

A virtual joystick to control motors, or a virtual switch to turn something on or off.

03

Manage settings on the phone

Save them to the device: set a thermostat temperature, or set an alarm clock.

04

Device state visualization

See the status of an LED (on/off) or the current servo angle.

MILESTONE REFLECTION · 2 OF 5

After connectivity

Complete your milestone reflection for this phase of the course before you leave today.

go.tufts.edu/reflect

BEFORE YOU GO

Questions?

Then get your device online. Next week: Final Robot Assembly and Testing — integrate everything you've learned into a working robot.

Individual assignment: electronics project — present a working circuit.

WHERE TO FIND US

Office hours

go.tufts.edu/meet_tvdv

Email

thomas.van_de_velde@tufts.edu

Course materials

tuftsmaker.github.io/ENT-164